

CodeGraph — Re-Benchmark After Fixes

2026-06-10 · CHARLIE · v0.9.5 · vs. eval 2026-06-09 · PR #1869

Verdict: the fix worked. The silent call-graph corruption is gone. Every Python symbol that returned **0 / wrong** callers on the stale index now returns the **correct** callers — confirmed on both the CLI and the live in-session daemon.

Grades

Axis	Before (as run)	After fix
Symbol coverage	A (9/9)	A (9/9)
Python call-graph (as operated)	D — silent corruption	A– — correct & kept fresh
Freshness plumbing	C– — hooks dead	A– — hooks + daily reindex + canary
Performance	A (6.9s)	A (9.2s rebuild)

The proof — callers before vs after

Symbol	Stale index	Now
<code>resolve_uns_path</code>	0	20
<code>_should_fire_uns_gate</code> (unique method)	0	9
<code>_maybe_dispatch_via_dst</code> (unique method)	0	2
<code>_make_result</code>	wrong twin	20
<code>withTenantContext</code> / <code>session0r401</code> (TS)	20	20 (stable)

The two unique-named methods are the clean control — no name collision is possible, so 0→9 and 0→2 proves a genuine repair, not luck.

What actually changed

- The fix is **operational, not algorithmic**. CodeGraph was always accurate on a *fresh* index; the watcher silently corrupted it over time.
- Now the index is **kept** fresh: daily `index --force` (9s), a self-healing canary on every merge, and the git hooks finally wired.
- Worst-case staleness is bounded to $\leq 24\text{h}$, and any merge repairs it immediately.

Still limited (upstream — grep covers these)

- `Supervisor` = 0 callers — class instantiation isn't an edge (upstream issue #774).
- `check_citation_compliance` = 0 — called via an import alias; neither name resolves. (New finding: the original eval was wrong to say this "resolves on fresh".)
- Same-name symbols can't be scoped to one definition.

Full report: [docs/tech-debt/2026-06-10-codegraph-rebenchmark.md](#) · Fix shipped in PR #1869 · upstream bugs filed: [colbymchenry/codegraph #773](#), [#774](#)